

BUZZ/1.0

Quiz Buzzer Protocol — Application-Layer Protocol Specification

1. วัตถุประสงค์ของโปรแกรม (Objective)

BUZZ/1.0 คือ network application สำหรับจำลองระบบ “กดออกตอบคำถามแบบเรียลไทม์” ที่มีผู้แข่งขันหลายคนแข่งกันกดปุ่มเพื่อชิงสิทธิ์ตอบคำถามก่อน โดยมี Server เป็นตัวกลางตัดสินว่าใครกดก่อน และ มี Host เป็น client อีกประเภทหนึ่งที่คอยควบคุมเกม

2. คุณลักษณะของโปรแกรม (Characteristics)

- Client-Server architecture โดย Client มี 2 บทบาท (role) ที่ใช้โปรแกรมเดียวกันแต่ต่างโหมด: Player ลงทะเบียนเข้าเกม, กด buzz, ส่งคำตอบ และ Host ควบคุมเกม ส่งคำถามถัดไป และตัดสินคำตอบ
- Real-time / low-latency** ผลของการกดออกต้องถึง server และแจกจ่ายกลับไปยังผู้เล่นทุกคนโดยเร็วที่สุด เพราะเกี่ยวข้องกับความเร็วในการตอบสนอง
- Broadcast state synchronization** เมื่อมีเหตุการณ์สำคัญเกิดขึ้น (คำถามใหม่, มีคนกด buzz ได้ก่อน, ผลการตัดสิน) server จะ broadcast สถานะไปยัง client ทุกคนพร้อมกัน เพื่อให้ทุกคนเห็นสถานะเกมตรงกัน
- Stateful server** ให้ server เก็บสถานะของเกมไว้ทั้งหมด (รายชื่อผู้เล่น, คะแนน, คำถามปัจจุบัน, ใครถือสิทธิ์ตอบอยู่) ฝั่ง client แทบไม่ต้องเก็บ state อะไรนอกจาก player id ของตัวเอง
- Text-based request/response protocol** คล้ายรูปแบบของ HTTP (มี status code + status phrase) ทำให้อ่าน debug ง่ายด้วยตาเปล่า

3. Transport Layer ที่เลือกใช้ และเหตุผล

เลือกใช้ UDP (User Datagram Protocol) ด้วยเหตุผลดังนี้:

- Latency** สำคัญกว่าความน่าเชื่อถือของทุกแพ็กเก็ต จุดสำคัญที่สุดของเกมนี้คือ “ใครกด buzz มาถึง server ก่อน” การที่ TCP ต้องทำ 3-way handshake, ควบคุม congestion, และ retransmission เมื่อแพ็กเก็ตหาย จะเพิ่ม latency และอาจทำให้ลำดับการมาถึงของ BUZZ message คลาดเคลื่อนจากเวลาที่ผู้เล่นกดจริง ซึ่งกระทบความเป็นธรรมของเกมโดยตรง
- Packet loss** บางครั้งไม่ใช่ปัญหาร้ายแรง ถ้าแพ็กเก็ต REGISTER หรือ BUZZ หายไป ผู้เล่นสามารถกดซ้ำ หรือ client ออกแบบให้ตรวจสอบและส่งใหม่ได้เอง (application-level reliability) โดยไม่ต้องแบกรับภาระของ TCP ที่ควบคุมทุกไบต์ให้ครบ
- รูปแบบการสื่อสารเป็นแบบ **message-oriented** อยู่แล้ว แต่ละ request/response ของ BUZZ/1.0 เป็นข้อความเดี่ยว ๆ จบในตัว (self-contained) ไม่ได้ต้องการ stream ต่อเนื่องแบบ TCP ทำให้ UDP ซึ่งเป็น datagram-oriented เหมาะสมกว่า
- รองรับ **broadcast** ไปยังหลาย client ได้ตรงไปตรงมา server ส่ง state update ไปยังผู้เล่นทุกคนด้วยการ sendto() วนซ้ำแบบง่าย โดยไม่ต้องคง connection (socket) แยกต่อ client เหมือนกรณีใช้ TCP

4. Application-Layer Protocol Specification

4.1 รูปแบบข้อความทั่วไป (General Message Format)

```
Request : BUZZ/1.0 <SEQ> <COMMAND> <field1> <field2> ...
Response : BUZZ/1.0 <SEQ> <STATUS_CODE> <STATUS_PHRASE> <field1> <field2> ...
```

- SEQ — เลขลำดับข้อความ (สำหรับ request คือเลขที่ client กำหนดเอง, สำหรับ response ที่เป็น broadcast จะเป็นเลขที่ server นับเอง)
- ฟิลด์สุดท้าย (เช่น ข้อความคำถาม หรือคำตอบ) อาจมีช่องว่างได้ และจะถูกอ่านเป็น “ส่วนที่เหลือทั้งหมดของข้อความ” (คล้าย maxsplit ใน Python)

4.2 Client → Server Requests

Command	รูปแบบ	คำอธิบาย
REGISTER	BUZZ/1.0 <seq> REGISTER <name>	ผู้เล่นลงทะเบียนเข้าเกมด้วยชื่อเล่น
BUZZ	BUZZ/1.0 <seq> BUZZ <player_id>	ผู้เล่นกดอดชิงสิทธิ์ตอบคำถามปัจจุบัน
ANSWER	BUZZ/1.0 <seq> ANSWER <player_id> <text>	ส่งคำตอบ (ต้องเป็นคนที่ buzz ได้ก่อนเท่านั้น)
HOSTNEXT	BUZZ/1.0 <seq> HOSTNEXT <hostkey>	Host สั่งเปิดคำถามถัดไป
HOSTJUDGE	BUZZ/1.0 <seq> HOSTJUDGE <hostkey> <CORRECT/WRONG>	Host ตัดสินคำตอบของผู้ที่ buzz ได้
HOSTRESET	BUZZ/1.0 <seq> HOSTRESET <hostkey>	Host เปิด buzzer ใหม่โดยไม่เปลี่ยนคำถาม

4.3 Server → Client Responses / Broadcasts

Code	Phrase	Cast	ความหมาย
200	OK REGISTERED	Unicast	ลงทะเบียนสำเร็จ พร้อม player_id ที่ server กำหนดให้
201	OK NEW_QUESTION	Broadcast	คำถามใหม่ถูกเปิด พร้อมเลขคำถามและเนื้อคำถาม
210	OK BUZZ_WINNER	Broadcast	ประกาศว่าใครกด buzz ได้ก่อน (buzzer ถูก lock ทันที)
250	OK ANSWER_RECEIVED	Broadcast	แจ้งว่าผู้เล่นที่ buzz ได้ส่งคำตอบอะไรมา
300	OK RESULT_CORRECT	Broadcast	คำตอบถูกต้อง พร้อมคะแนนรวมล่าสุด
301	OK RESULT_WRONG	Broadcast	คำตอบผิด
350	OK BUZZER_OPEN	Broadcast	เปิด buzzer ใหม่ให้กดได้อีกครั้ง

Code	Phrase	Cast	ความหมาย
390	OK NO_MORE_QUESTIONS	Broadcast	คำถามหมดแล้ว จบเกม
400	ERROR INVALID_FORMAT / NOT_YOUR_TURN	Unicast	คำสั่งผิดรูปแบบ หรือไม่ใช่ตาของผู้ส่ง
401	ERROR UNAUTHORIZED	Unicast	hostkey ไม่ถูกต้อง
404	ERROR UNKNOWN_PLAYER	Unicast	ไม่พบ player_id ที่ระบุ
420	LOCKED ALREADY_BUZZED	Unicast	ปฎิเสธ เพราะมีคนอื่นกดก่อนแล้ว
430	LOCKED NO_ACTIVE_QUESTION	Unicast	ปฎิเสธ เพราะยังไม่มีคำถามเปิดอยู่

4.4 ตัวอย่างลำดับการสื่อสาร (Message Sequence Example)

```

Alice -> Server : BUZZ/1.0 1 REGISTER Alice
Server -> Alice : BUZZ/1.0 1 200 OK REGISTERED 1 Alice

Bob   -> Server : BUZZ/1.0 1 REGISTER Bob
Server -> Bob   : BUZZ/1.0 1 200 OK REGISTERED 2 Bob

Host  -> Server : BUZZ/1.0 1 HOSTNEXT admin123
Server -> ALL   : BUZZ/1.0 1 201 OK QUESTION 0 How much is 3 plus 5?

Alice -> Server : BUZZ/1.0 2 BUZZ 1
Server -> ALL   : BUZZ/1.0 2 210 OK BUZZ_WINNER 1 Alice

Bob   -> Server : BUZZ/1.0 2 BUZZ 2
Server -> Bob   : BUZZ/1.0 2 420 LOCKED ALREADY_BUZZED Alice

Alice -> Server : BUZZ/1.0 3 ANSWER 1 8
Server -> ALL   : BUZZ/1.0 3 250 OK ANSWER_RECEIVED Alice 8

Host  -> Server : BUZZ/1.0 2 HOSTJUDGE admin123 CORRECT
Server -> ALL   : BUZZ/1.0 4 300 OK RESULT_CORRECT Alice SCORE 10

```

5. State Diagram (per question)

```

NO_ACTIVE_QUESTION --(HOSTNEXT)--> QUESTION_OPEN
QUESTION_OPEN --(BUZZ ครั้งแรก)--> LOCKED (มีคนถือสิทธิ์ตอบ)
LOCKED --(HOSTJUDGE CORRECT)--> NO_ACTIVE_QUESTION (รอ HOSTNEXT ข้อถัดไป)
LOCKED --(HOSTJUDGE WRONG)--> QUESTION_OPEN (buzzer เปิดใหม่ ข้อเดิม)
LOCKED --(HOSTRESET)--> QUESTION_OPEN

```

6. วิธี Build และรัน

```
make                # compile ทั้งคู่
./buzz_server       # เปิด server (terminal 1)
./buzz_client --host admin123  # เปิด host (terminal 2)
./buzz_client --name Alice    # เปิด client ผู้เล่น (terminal 3)
./buzz_client --name Bob      # เปิด client ผู้เล่นอีกคน (terminal 4)
```

ในฝั่ง Host พิมพ์คำสั่ง: next, correct, wrong, reset, quit

ในฝั่ง Player พิมพ์: buzz เพื่อกดออก หรือพิมพ์คำตอบแล้ว Enter เพื่อส่งคำตอบ